Forecasting Stock Market Volatility Using Daily S&P 500 ETF Returns

He Jiang, Patrick Zhu

2026-04-05

```r
library(quantmod)
library(zoo)
library(xts)
library(readr)
library(dplyr)
library(ggplot2)
library(forecast)
```

Introduction

This project studies stock market volatility using daily S&P 500 ETF returns. Instead of forecasting raw stock prices, we focus on volatility because financial returns often display volatility clustering and are more appropriate for time series modeling.

Data Download and Preparation

We use daily data for the SPDR S&P 500 ETF Trust (SPY) downloaded from Yahoo Finance. Adjusted closing prices are used to compute daily log returns, and volatility is measured as the 21-day rolling standard deviation of returns.

```r
# Download SPY data from Yahoo Finance
getSymbols("SPY", src = "yahoo", from = "2015-01-01")
```

```
## [1] "SPY"
```

```r
# Extract adjusted close price
spy_adj <- Ad(SPY)

# Compute daily log returns
spy_ret <- diff(log(spy_adj))

# Compute 21-day rolling volatility
spy_vol_21 <- rollapply(
  spy_ret,
  width = 21,
  FUN = sd,
```

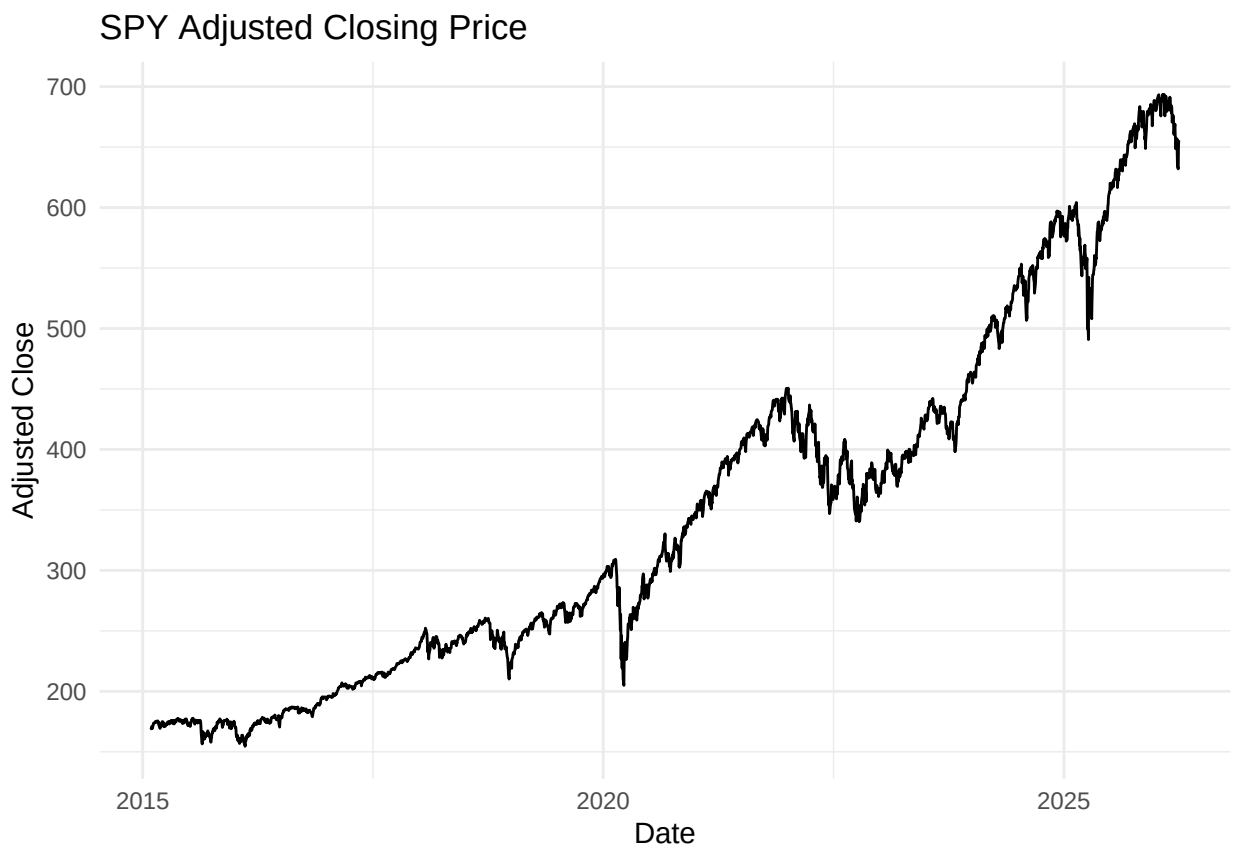
```r
  align = "right",
  fill = NA
)

# Build data frame
data <- data.frame(
  date = as.Date(index(spy_vol_21)),
  adj_close = as.numeric(spy_adj[index(spy_vol_21)]),
  return = as.numeric(spy_ret[index(spy_vol_21)]),
  volatility_21 = as.numeric(spy_vol_21)
)

data <- na.omit(data)

head(data)
```

```
##          date adj_close       return volatility_21
## 22 2015-02-03  169.6364  0.014357399   0.011266096
## 23 2015-02-04  168.9905 -0.003814940   0.010523720
## 24 2015-02-05  170.6964  0.010044501   0.010456468
## 25 2015-02-06  170.2244 -0.002769422   0.010185598
## 26 2015-02-09  169.4625 -0.004485555   0.009475872
## 27 2015-02-10  171.2679  0.010597003   0.009586021
```

Exploratory Data Analysis

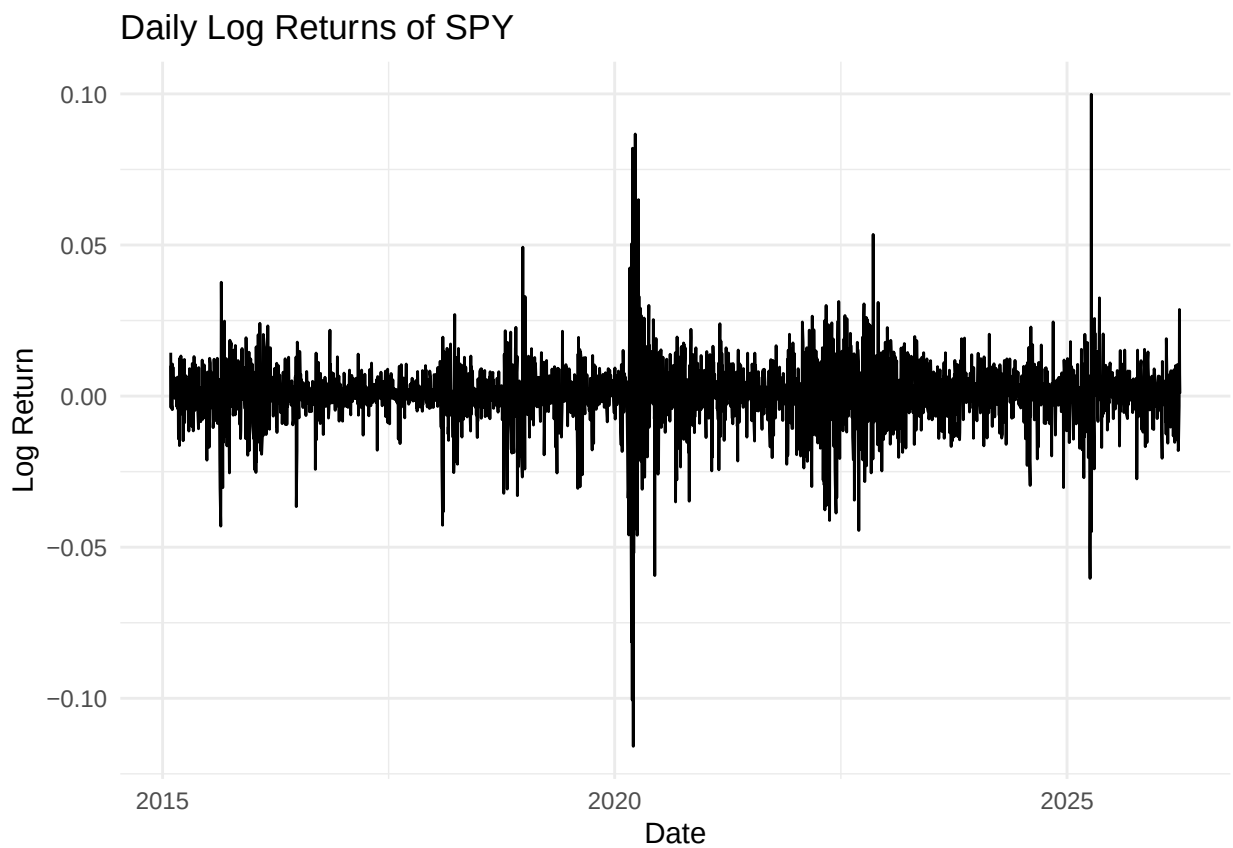
SPY Adjusted Close

```r
ggplot(data, aes(x = date, y = adj_close)) +
  geom_line() +
  labs(title = "SPY Adjusted Closing Price", x = "Date", y = "Adjusted Close") +
  theme_minimal()
```



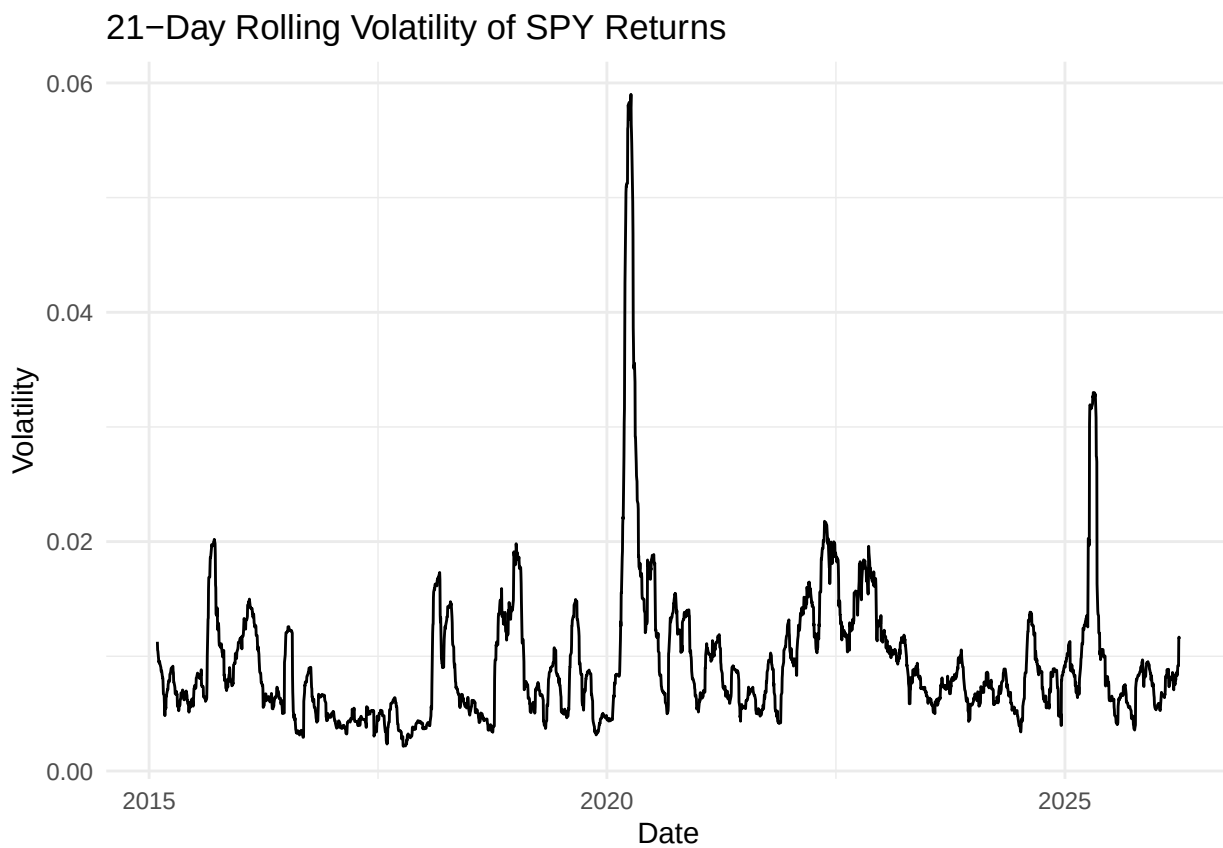
Daily Returns

```
ggplot(data, aes(x = date, y = return)) +
  geom_line() +
  labs(title = "Daily Log Returns of SPY", x = "Date", y = "Log Return") +
  theme_minimal()
```



21-Day Rolling Volatility

```
ggplot(data, aes(x = date, y = volatility_21)) +
  geom_line() +
  labs(title = "21-Day Rolling Volatility of SPY Returns", x = "Date", y = "Volatility") +
  theme_minimal()
```



Preliminary Time Series Analysis

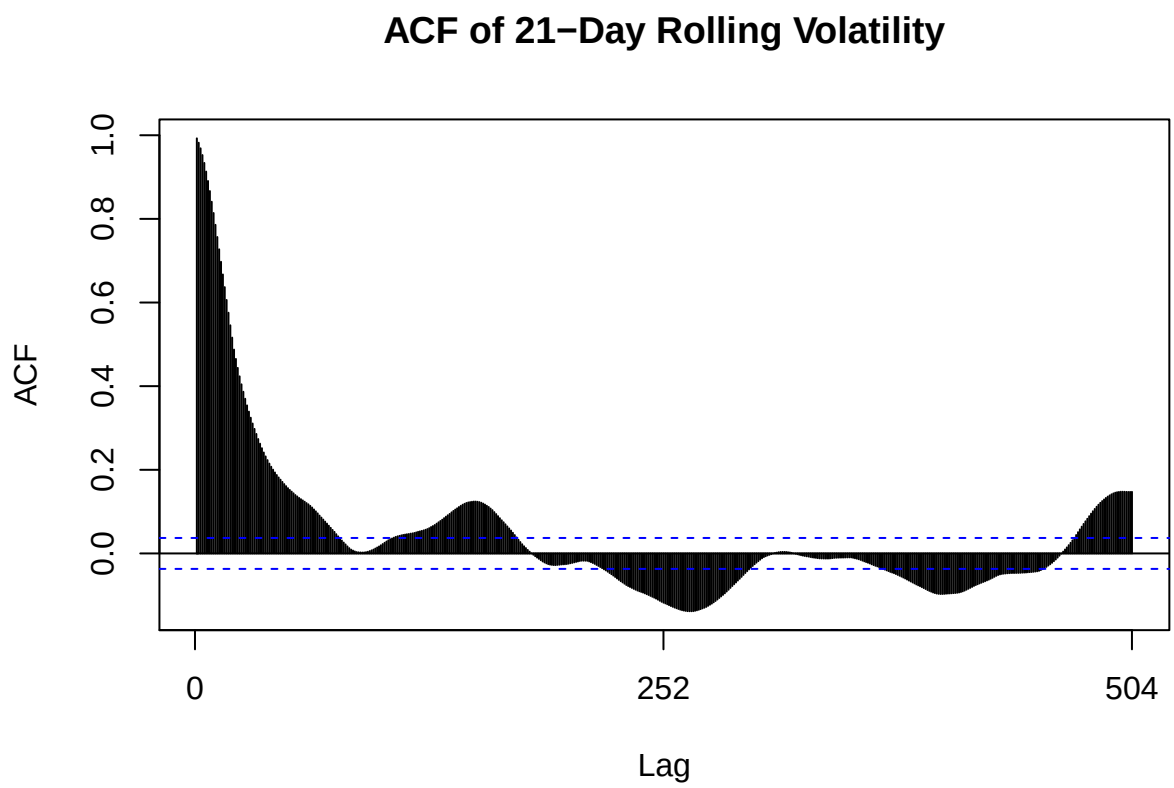
Summary Statistics

```
summary(data)
```

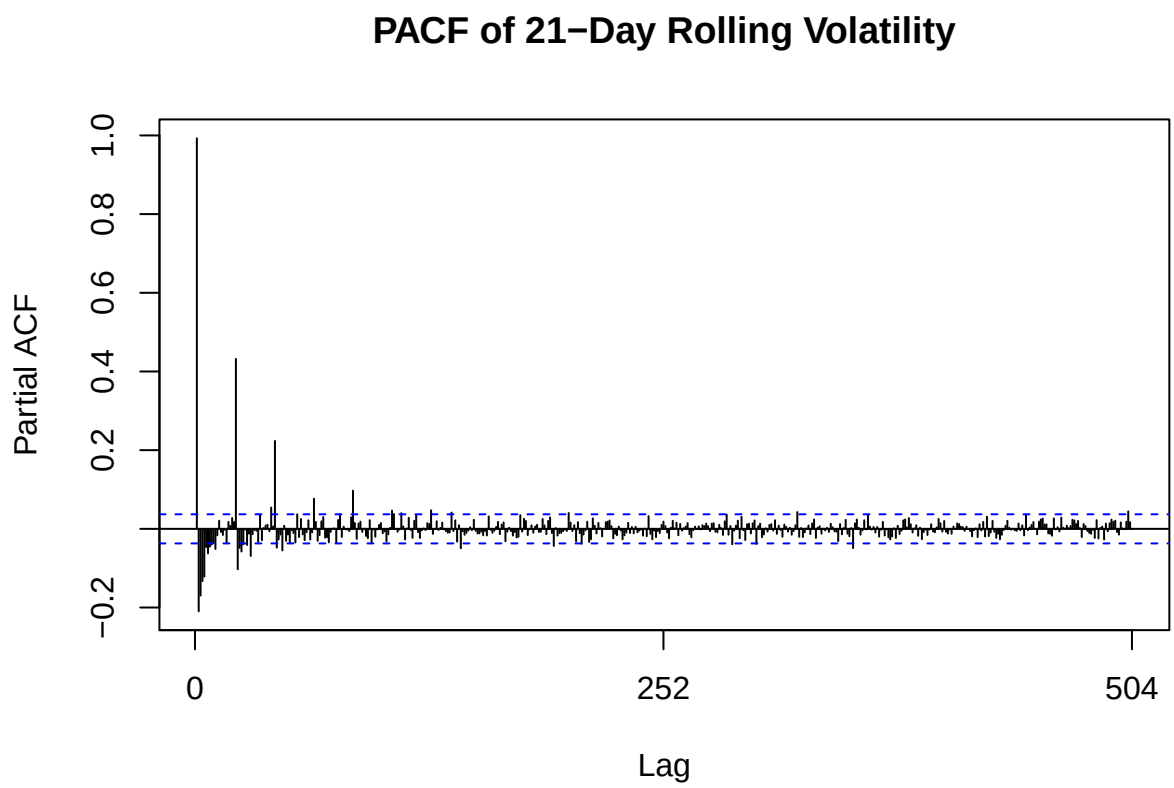
```
##       date              adj_close         return           volatility_21
##  Min.   :2015-02-03   Min.   :154.6   Min.   :-0.1158866   Min.   :0.002152
##  1st Qu.:2017-11-13   1st Qu.:224.2   1st Qu.:-0.0036831   1st Qu.:0.005781
##  Median :2020-08-29   Median :309.3   Median : 0.0006118   Median :0.007775
##  Mean   :2020-08-30   Mean   :345.9   Mean   : 0.0004867   Mean   :0.009365
##  3rd Qu.:2023-06-14   3rd Qu.:428.0   3rd Qu.: 0.0058977   3rd Qu.:0.011423
##  Max.   :2026-04-02   Max.   :693.6   Max.   : 0.0998627   Max.   :0.059005
```

ACF and PACF of Volatility

```
vol_ts <- ts(data$volatility_21, frequency = 252)
Acf(vol_ts, main = "ACF of 21-Day Rolling Volatility")
```




```
Pacf(vol_ts, main = "PACF of 21-Day Rolling Volatility")
```



Next Steps

The next stage of the project will split the data into training and test sets and compare forecasting performance across multiple models, such as ARIMA and GARCH.

Train/Test Split

We split the volatility series into a training set and a test set. The training set is used to fit the models, and the test set is used to evaluate out-of-sample forecasting performance.

```
n <- nrow(data)
train_size <- floor(0.8 * n)

train_data <- data[1:train_size, ]
test_data  <- data[(train_size + 1):n, ]

train_vol <- train_data$volatility_21
test_vol  <- test_data$volatility_21

length(train_vol)
```

```
## [1] 2246
```

```r
length(test_vol)
```

```
## [1] 562
```

```r
tail(train_data)
```

```
##            date adj_close        return volatility_21
## 2262 2023-12-27  463.7478  0.0018065030   0.005743097
## 2263 2023-12-28  463.9230  0.0003775917   0.005752407
## 2264 2023-12-29  462.5800 -0.0028991134   0.005831377
## 2265 2024-01-02  459.9912 -0.0056121037   0.006066125
## 2266 2024-01-03  456.2346 -0.0082001234   0.006369268
## 2267 2024-01-04  454.7650 -0.0032263586   0.006281018
```

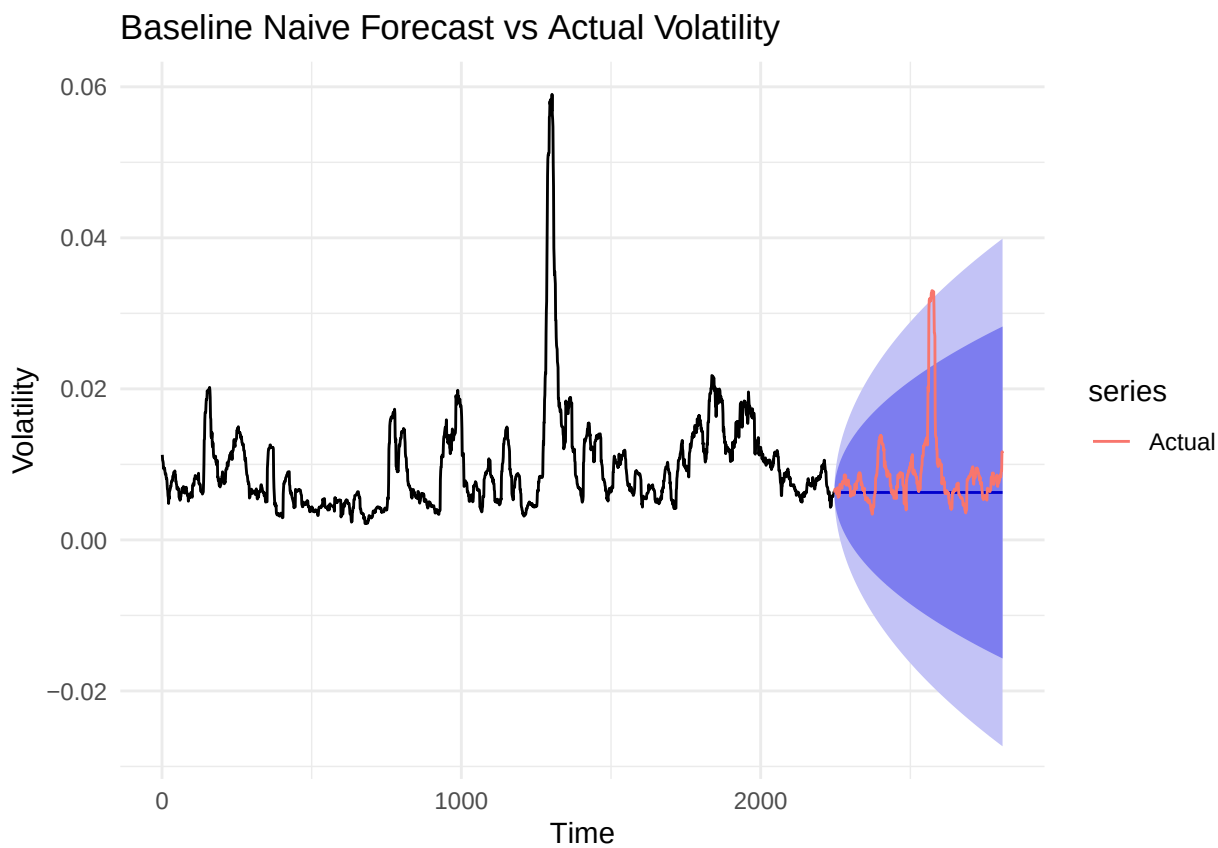
```r
head(test_data)
```

```
##            date adj_close        return volatility_21
## 2268 2024-01-05  455.3879  0.0013688356   0.006271841
## 2269 2024-01-08  461.8890  0.0141748829   0.006730719
## 2270 2024-01-09  461.1883 -0.0015182137   0.006660397
## 2271 2024-01-10  463.7964  0.0056394267   0.006692140
## 2272 2024-01-11  463.5921 -0.0004406906   0.006693191
## 2273 2024-01-12  463.9132  0.0006924736   0.006662333
```

Baseline Forecast

As a simple benchmark, we first use a naive forecasting approach. This baseline helps us evaluate whether more advanced models such as ARIMA and GARCH actually improve forecasting accuracy.

```r
train_ts <- ts(train_vol, start = 1, frequency = 1)
test_ts  <- ts(test_vol,  start = length(train_ts) + 1, frequency = 1)

baseline_fit <- naive(train_ts, h = length(test_ts))

autoplot(baseline_fit) +
  autolayer(test_ts, series = "Actual") +
  labs(
    title = "Baseline Naive Forecast vs Actual Volatility",
    x = "Time",
    y = "Volatility"
  ) +
  theme_minimal()
```




```r
accuracy(baseline_fit, test_ts)
```

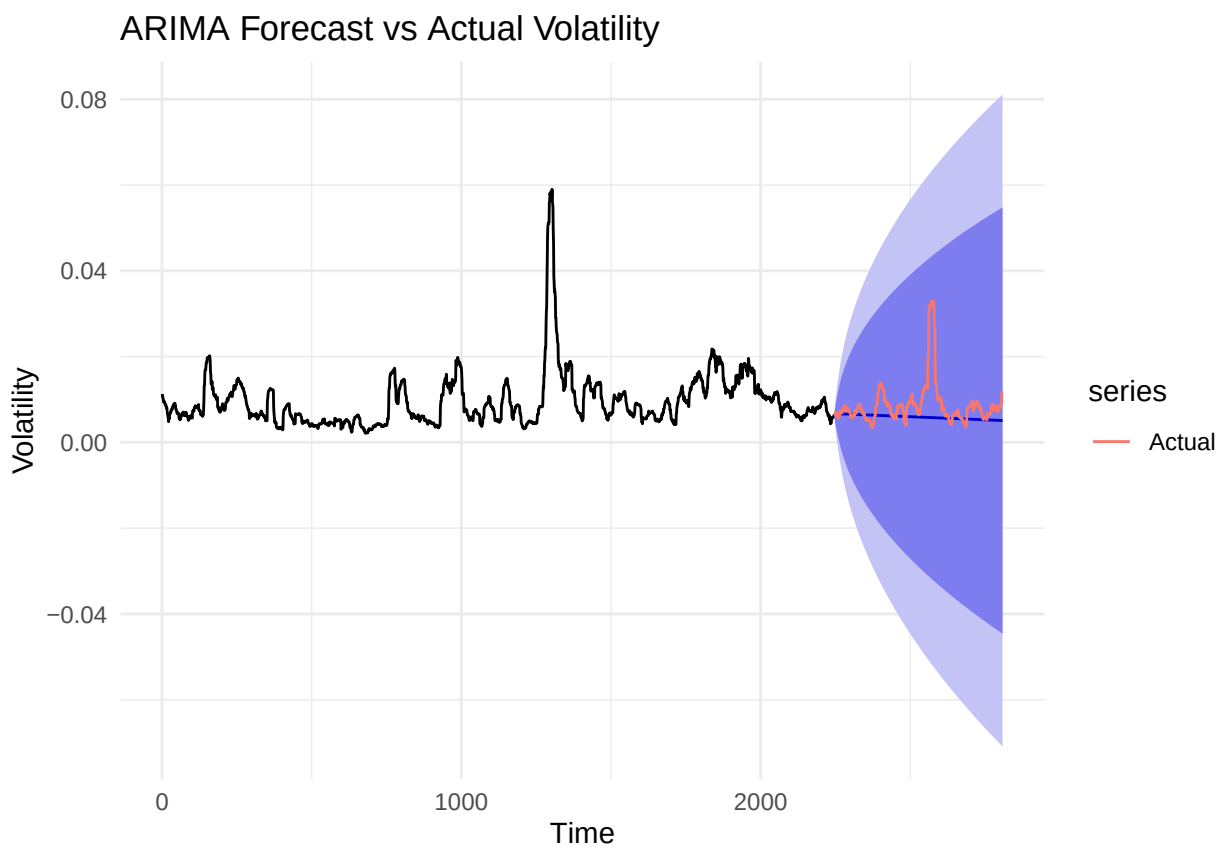
```
##                         ME         RMSE          MAE        MPE      MAPE
## Training set -2.220525e-06 0.0007234047 0.0003549931 -0.2804592  3.867874
## Test set      2.454140e-03 0.0056462108 0.0029513616 15.2656723 26.035806
##                  MASE      ACF1 Theil's U
## Training set 1.000000 0.2228754        NA
## Test set     8.313855 0.9849643  3.701732
```

```r
arima_fit <- auto.arima(train_ts, seasonal = FALSE)
summary(arima_fit)
```

```
## Series: train_ts
## ARIMA(3,1,2) with drift
##
## Coefficients:
##           ar1     ar2     ar3     ma1      ma2  drift
##       -0.1350  0.8373  0.0356  0.2707  -0.6339      0
## s.e.   0.0533  0.0275  0.0295  0.0494   0.0424      0
##
## sigma^2 = 4.598e-07:  log likelihood = 13197.28
## AIC=-26380.56   AICc=-26380.51   BIC=-26340.54
##
## Training set error measures:
##                         ME         RMSE          MAE         MPE     MAPE
```

```
## Training set 4.828995e-07 0.0006770597 0.0003653633 0.002847927 4.051831
##                   MASE         ACF1
## Training set 1.029212 0.0006150374
```

```
arima_fc <- forecast(arima_fit, h = length(test_ts))

autoplot(arima_fc) +
  autolayer(test_ts, series = "Actual") +
  labs(
    title = "ARIMA Forecast vs Actual Volatility",
    x = "Time",
    y = "Volatility"
  ) +
  theme_minimal()
```




```
accuracy(arima_fc, test_ts)
```

```
##                        ME         RMSE          MAE          MPE      MAPE
## Training set 4.828995e-07 0.0006770597 0.0003653633  0.002847927  4.051831
## Test set     2.834575e-03 0.0058628909 0.0032105036 20.298033091 28.527090
##                  MASE         ACF1 Theil's U
## Training set 1.029212 0.0006150374        NA
## Test set     9.043847 0.9849860660  3.953332
```

```r
baseline_acc <- accuracy(baseline_fit, test_ts)
arima_acc    <- accuracy(arima_fc, test_ts)

comparison <- data.frame(
  Model = c("Naive Baseline", "ARIMA"),
  RMSE  = c(baseline_acc[2, "RMSE"], arima_acc[2, "RMSE"]),
  MAE   = c(baseline_acc[2, "MAE"],  arima_acc[2, "MAE"]),
  MAPE  = c(baseline_acc[2, "MAPE"], arima_acc[2, "MAPE"])
)

comparison
```

```
##            Model        RMSE         MAE     MAPE
## 1 Naive Baseline 0.005646211 0.002951362 26.03581
## 2          ARIMA 0.005862891 0.003210504 28.52709
```